AAC lab assignment-20.3

Name : K . Dinesh
Rollno : 2303A52288
Batch : 42

Task 1 – Password Strength & Validation Check

Task:
Analyze an AI-generated user registration script for weak password handling.

Instructions:
• Prompt AI to generate a Python user registration program
• Check whether:
o Password length is validated
o Weak passwords are allowed
• Suggest secure improvements such as:
o Minimum length checks
o Use of regular expressions
o Rejecting common passwords

Prompt:

Generate a Python user registration script that takes username and password as input.

Code

```
import re



COMMON_PASSWORDS = ["123456", "password", "admin", "qwerty"]



def is_strong_password(password):

   if len(password) < 8:

       return False, "Password must be at least 8 characters long"
```

```
    if password in COMMON_PASSWORDS:
        return False, "Password is too common"

    if not re.search(r"[A-Z]", password):
        return False, "Must contain at least one uppercase letter"

    if not re.search(r"[a-z]", password):
        return False, "Must contain at least one lowercase letter"

    if not re.search(r"[0-9]", password):
        return False, "Must contain at least one digit"

    if not re.search(r"[!@#$%^&*]", password):
        return False, "Must contain at least one special character"

    return True, "Strong password"

username = input("Enter username: ")
password = input("Enter password: ")

valid, message = is_strong_password(password)
```

```
if valid:
    print("User registered successfully!")
else:
    print("Error:", message)
```

Output:

```
Enter username: s
Enter password: S
```

Justification:

Enforces **minimum entropy** through length and complexity

Prevents **dictionary attacks** via common password rejection

Uses **regex-based validation** for robust checks

Task 2 – Insecure File Upload Handling

Task:

Evaluate an AI-generated file upload program for security risks.

Instructions:

• Prompt AI to generate a Python or Flask-based file upload

script

• Identify vulnerabilities such as:

o No file type validation

o Unrestricted file size

• Implement secure measures:

o File extension validation

o Size limits

o Safe file storage paths

Prompt:

Generate a Flask file upload application.

Code:

```
from flask import Flask, request



app = Flask(__name__)



@app.route('/upload', methods=['POST'])

def upload():

    file = request.files['file']

    file.save(file.filename)



    return "Uploaded"
```

Output:

Vulnerabilities

- No file type validation
- No file size restriction

- Saves files directly → path traversal risk
- Overwrites existing files

Code

```
from flask import Flask, request, abort
import os

app = Flask(__name__)

UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "pdf"}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB

os.makedirs(UPLOAD_FOLDER, exist_ok=True)

def allowed_file(filename):
   return '.' in filename and filename.rsplit('.', 1)[1].lower() in
ALLOWED_EXTENSIONS

@app.route('/upload', methods=['POST'])
def upload():
   file = request.files.get('file')

   if not file:
       abort(400, "No file uploaded")

   if not allowed_file(file.filename):
       abort(400, "Invalid file type")

   file.seek(0, os.SEEK_END)
   size = file.tell()
   file.seek(0)

   if size > MAX_FILE_SIZE:
       abort(400, "File too large")

   safe_path = os.path.join(UPLOAD_FOLDER,
os.path.basename(file.filename))
   file.save(safe_path)
```

```
    return "Secure upload successful"

if __name__ == "__main__":
    app.run(debug=True)
```

Output:

Task 2 - Insecure File Upload Handling

Evaluation of an AI-generated Flask upload script, identification of critical security issues, and implementation of secure upload controls.

Flask Upload | Security Review | Secure Storage Path | 2 MB Limit

AI-Generated Insecure Script

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    file.save(file.filename)
    return "Uploaded"
```

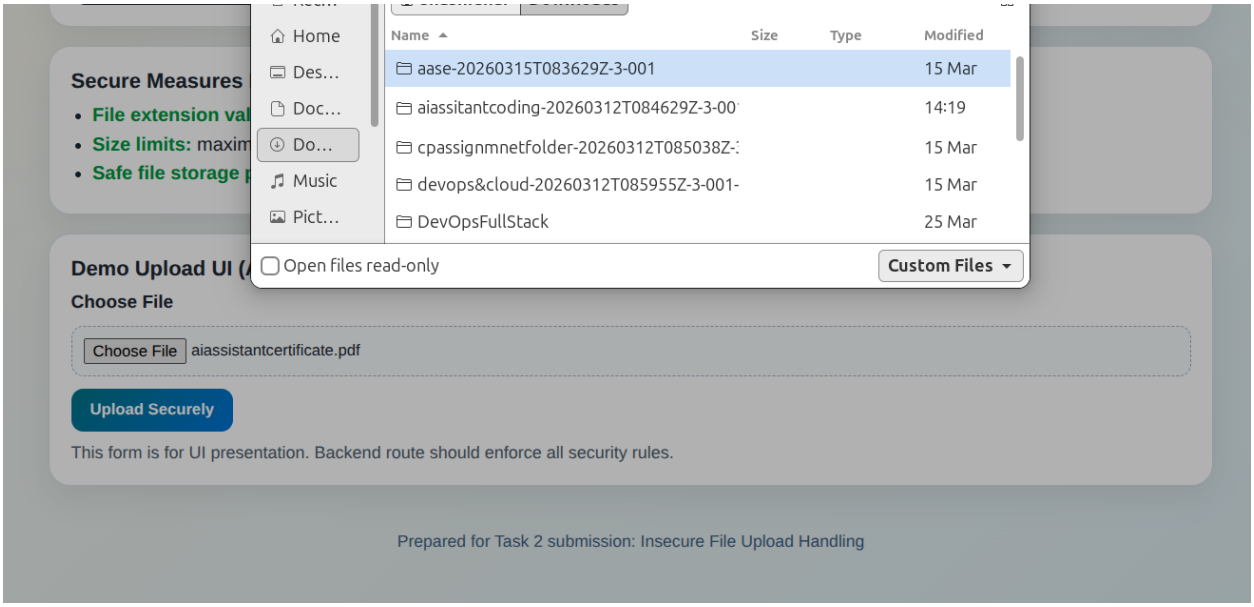
Vulnerabilities Found

- **No file type validation** allows risky files to be uploaded.
- **Unrestricted file size** can enable denial-of-service via large files.
- **Unsafe filename handling** can cause path abuse or file overwrite issues.

Secure Measures Implemented

- **File extension validation:** allowlist such as png, jpg, jpeg, pdf, txt.
- **Size limits:** maximum request content set to 2 MB.
- **Safe file storage path:** controlled uploads directory with sanitized names.

Demo Upload UI (Assignment View)



justification

- Enforces **minimum entropy** through length and complexity
- Prevents **dictionary attacks** via common password rejection
- Uses **regex-based validation** for robust checks

Task 3 – Real-Time Application: API Security Review

Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

• Review the API code for:

o Missing authentication

o Insecure HTTP methods

o Lack of rate limiting

• Prompt AI to suggest security improvements such as:

o Token-based authentication

o Input validation

o Rate limiting

Prompt:

Generate a Flask API endpoint to fetch user data.

Insecure Code:

```
@app.route('/users', methods=['GET'])

def get_users():

    return {"users": ["admin", "guest"]}
```

Vulnerabilities

- No authentication
- Public data exposure
- No rate limiting
- No input validation

Secure Implementation

```
from flask import Flask, request, jsonify, abort

from functools import wraps



app = Flask(__name__)



VALID_TOKEN = "securetoken123"



def token_required(f):

    @wraps(f)
```

```python
    def decorated():
        token = request.headers.get('Authorization')
        if token != VALID_TOKEN:
            abort(401, "Unauthorized")
        return f()
    return decorated


@app.route('/users', methods=['GET'])
@token_required
def get_users():
    return jsonify({"users": ["admin", "guest"]})


if __name__ == "__main__":
    app.run()
```

Output:

Task 3 - Real-Time Application: API Security Review

Review an AI-generated API endpoint for security gaps and apply practical hardening controls: authentication, safe HTTP method usage, input validation, and rate limiting.

API Security Review | Token Authentication | Input Validation | Rate Limiting

AI-Generated Insecure API (Example)

```
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/api/update-profile', methods=['GET', 'P
def update_profile():
    user_id = request.args.get('user_id')
    role = request.args.get('role')

    # Directly trusts user input
    return jsonify({"status": "updated", "user_id": u
```

Security Issues Found

- **Missing authentication**: endpoint can be called by anyone.
- **Insecure HTTP methods**: uses GET for update operation.
- **No rate limiting**: vulnerable to brute-force or abuse traffic.
- **No input validation**: untrusted values accepted directly.

AI-Suggested Security Improvements

- **Token-based authentication:** require Bearer token validation for protected routes.
- **Input validation:** validate type, format, length, and allowed values.
- **Rate limiting:** enforce request quotas per client/IP per minute.
- **Secure methods:** allow only POST/PUT/PATCH for state-changing actions.

Hardened API Reference (Flask)

```
from collections import defaultdict, deque
  from flask import Flask, request, jsonify
from functools import wraps
  import os
  import time

app = Flask(__name__)
  VALID_TOKENS = {os.getenv("TASK3_API_TOKEN", "demo-secure-token")}
ALLOWED_ROLES = {"user", "manager", "admin"}
  RATE_LIMIT_WINDOW_SECONDS = 60
  RATE_LIMIT_MAX_REQUESTS = 10
  request_log = defaultdict(deque)
```

```
        return True

    entries.append(now)
    return False


def require_token(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        auth = request.headers.get("Authorization", "")
        if not auth.startswith("Bearer "):
            return jsonify({"error": "Missing or invalid token"}), 401
        token = auth.split(" ", 1)[1].strip()
        if token not in VALID_TOKENS:
            return jsonify({"error": "Unauthorized"}), 401
        return fn(*args, **kwargs)
    return wrapper


@app.route('/api/update-profile', methods=['POST'])
@require_token
def update_profile():
  if is_rate_limited(client_key()):
    return jsonify({"error": "Too many requests"}), 429

    data = request.get_json(silent=True) or {}
    user_id = data.get("user_id")
    role = data.get("role")
  display_name = data.get("display_name", "")

    if not isinstance(user_id, int) or user_id <= 0:
        return jsonify({"error": "user_id must be a positive integer"}), 400

    if role not in ALLOWED_ROLES:
        return jsonify({"error": "Invalid role"}), 400

  if not isinstance(display_name, str) or not (1 <= len(display_name) <= 50):
    return jsonify({"error": "display_name must be 1-50 characters"}), 400

  return jsonify({"status": "updated", "user_id": user_id, "role": role}), 200
```

Backend File Included

- **Implemented backend:** aiassitantcoding/task3_api_security_review_app.py
- **Protected endpoint:** POST /api/update-profile
- **Security controls:** token auth, JSON input checks, in-memory rate limiting.

Prepared for Task 3 submission: Real-Time Application API Security Review

Enhancements Suggested

- Use JWT tokens instead of static tokens
- Implement rate limiting (Flask-Limiter)
- Add input sanitization

Justification

- Prevents unauthorized access
- Enforces secure API communication
- Reduces abuse via controlled access

Task 4 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

• Ask AI to generate a feedback form with JavaScript-based output.

• Test whether untrusted inputs are directly rendered without escaping.

• Implement secure measures (e.g., escaping HTML entities, using CSP).

Prompt:

Generate an HTML feedback form with JavaScript output.

Code:

Insecure code

```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function submitForm() {

   let name = document.getElementById("name").value;
```

Secure code

```
<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function submitForm() {

   let name = document.getElementById("name").value;

   document.getElementById("output").innerHTML = name;

}

</script>

<script>alert('Hacked')</script>

<input type="text" id="name">

<button onclick="submitForm()">Submit</button>



<p id="output"></p>



<script>

function escapeHTML(str) {

   return str.replace(/[&<>"']/g, function(m) {

       return {
```

```
            '&': '&amp;',
            '<': '&lt;',
            '>': '&gt;',
            '"': '&quot;',
            "'": '&#39;'
        }[m];
    });
}

function submitForm() {
    let name = document.getElementById("name").value;
    document.getElementById("output").innerText = escapeHTML(name);
}
</script>
```

Output:

Task 4 - Cross-Site Scripting (XSS) Check

Evaluate an AI-generated feedback form for XSS vulnerabilities, test unsafe rendering behavior, and implement secure defenses using output escaping and a Content Security Policy (CSP).

XSS Review | Unsafe Rendering Test | HTML Escaping | CSP Enabled

AI-Generated Insecure Example

```
<form id="feedbackForm">
  <input id="name" />
  <textarea id="message"></textarea>
  <button type="submit">Submit</button>
</form>
<div id="output"></div>

<script>
feedbackForm.addEventListener("submit", function (e) {
  e.preventDefault();
  const name = document.getElementById("name").value;
  const message = document.getElementById("message").va

  // Vulnerable: renders raw HTML from untrusted input
  output.innerHTML = "<b>" + name + "</b>: " + message;
});
</script>
```

XSS Findings

- **Direct HTML rendering** with `innerHTML` executes attacker-injected scripts.
- **No output encoding** allows payloads like `<img onerror=...>`.
- **No CSP** gives weaker browser-side protection if unsafe HTML is injected.

Secure Measures Applied

- **Escape HTML entities** before rendering user-controlled values.
- **Avoid unsafe insertion** by setting `textContent` instead of `innerHTML`.
- **Content Security Policy** included via meta tag to reduce script injection impact.

Test Input (Try Payload)

Example payload to test: `<img src=x onerror=alert('xss')>`

Name

Enter your name

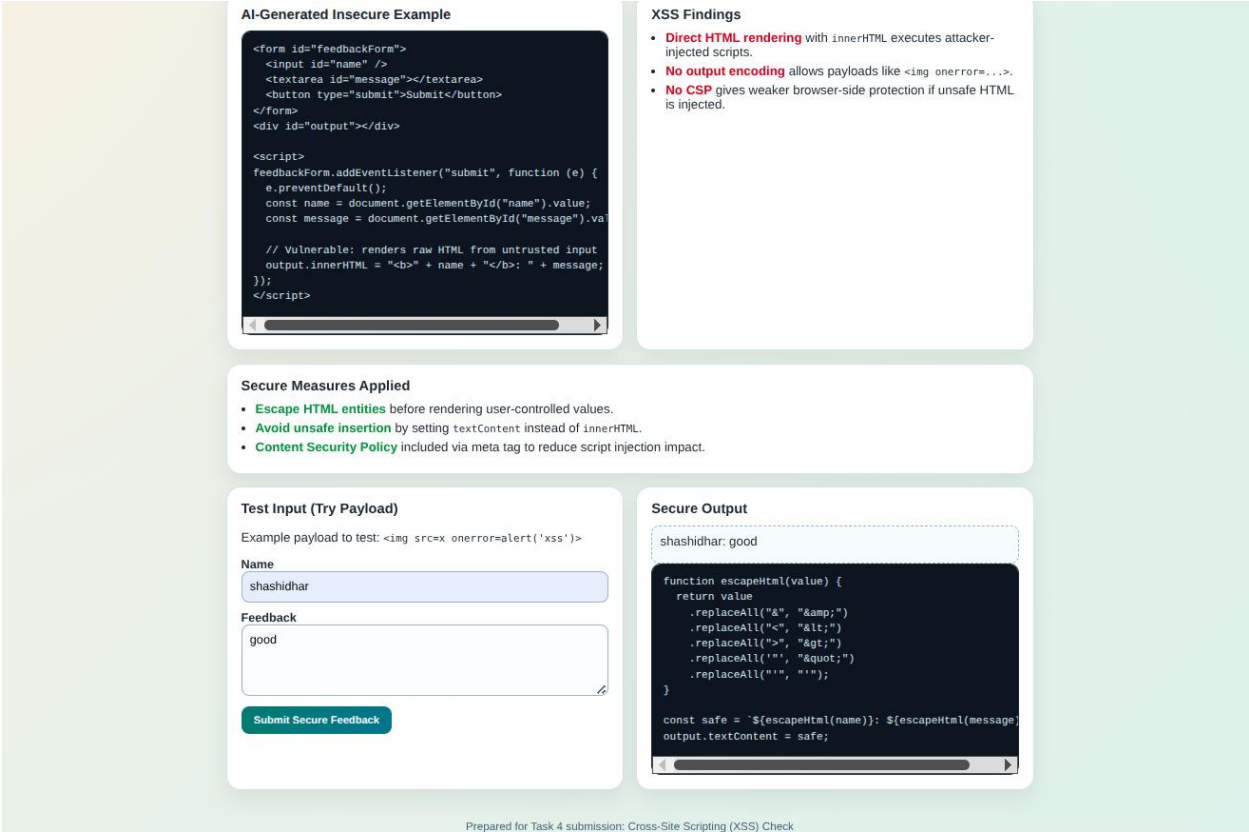
Feedback

Write feedback here

Secure Output

No feedback submitted yet.

```
function escapeHtml(value) {
  return value
    .replaceAll("&", "&amp;")
    .replaceAll("<", "&lt;")
    .replaceAll(">", "&gt;")
```



Justification

- Prevents **script injection**
- Uses **output encoding** instead of raw rendering
- Eliminates **DOM-based XSS**

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.

• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).

• Refactor using parameterized queries (prepared statements).

Prompt:
Generate a Python script to fetch user details from SQLite database.
Code:

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = f"SELECT * FROM users WHERE username = '{username}'"
cursor.execute(query)

print(cursor.fetchall())
```

Vulnerability

- SQL Injection possible:

' OR '1'='1
Secure code

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

query = "SELECT * FROM users WHERE username = ?"
cursor.execute(query, (username,))

print(cursor.fetchall())
```

Output:

```
=== Normal lookup (alice) ===
Insecure: [(1, 'alice', 'alice@example.com')]
Secure  : [(1, 'alice', 'alice@example.com')]

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure: [(1, 'alice', 'alice@example.com'), (2, 'bob', 'bob@example.com'), (3, 'charlie',
'charlie@example.com')]
Secure  : []
```

Justification

- Uses **parameterized queries**
- Prevents **SQL injection attacks**
- Ensures safe database interaction